```
)
Quantum Feature Map
feature_map = ZZFeatureMap(feature_dimension=2,
reps=2)
Quantum Kernel
from qiskit.primitives import Sampler
from qiskit_machine_learning.kernels import
FidelityQuantumKernel
from qiskit_machine_learning.state_fidelities import
ComputeUncompute

# Step 1: Create sampler
sampler = Sampler()

# Step 2: Create fidelity object
fidelity = ComputeUncompute(sampler=sampler)

# Step 3: Create quantum kernel
quantum_kernel = FidelityQuantumKernel(
    feature_map=feature_map,
    fidelity=fidelity
)
/tmp/ipykernel_1156/1101502941.py:9:
DeprecationWarning: V1 Primitives are deprecated as of
qiskit-machine-learning 0.8.0 and will be removed no
sooner than 4 months after the release date. Use V2
primitives for continued compatibility and support.
  fidelity = ComputeUncompute(sampler=sampler)
Train QSVM
# Use classical SVM with quantum kernel
qsvm = SVC(kernel=quantum_kernel.evaluate)

qsvm.fit(X_train, y_train)

accuracy_qsvm = qsvm.score(X_test, y_test)

print("QSVM Accuracy:", accuracy_qsvm)
QSVM Accuracy: 0.8859649122807017
```

Your pipeline is functionally correct, but there are **two technical issues** you should not ignore:

1. **Dependency conflict (NumPy 2.0 vs qctrl-visualizer)**
2. **This is not a “pure quantum model” — it is a classical SVM using a quantum kernel**

I'll explain both while walking through the system.

---

1. What you actually implemented

You built a **Quantum Kernel SVM (QSVM)** using:

- Classical optimizer → scikit-learn
- Quantum feature mapping → Qiskit Machine Learning

Core Idea

👉 Replace classical kernel with quantum kernel:

$$K(x_i, x_j) = |\langle \phi(x_i) | \phi(x_j) \rangle|^2$$

Meaning:

- Map data → quantum states $|\phi(x)\rangle$
- Kernel = similarity between quantum states

---

2. Step-by-Step Breakdown

🔹 Dataset

```python
X  y = datasets.                    return_X_y=True
X = X     2
```

Important observation (you should question this)

👉 You reduced **30 features → 2 features**

✔️ Required for quantum circuits
❗ But causes **information loss**

🔹 Normalization

```python
X =  X - X.       /  X.       - X.
```

✔ Needed because:

- Quantum rotations expect values in $[0, 2\pi]$ range

🔹 Feature Map

```python
ZZFeatureMap feature_dimension=2  reps=2
```

What it does:

- Encodes classical data into quantum states
- Adds **entanglement + non-linearity**

👉 Internally applies:

- Z rotations
- ZZ interactions between qubits

🔹 Quantum Kernel

```python
FidelityQuantumKernel
```

How it works:

1. Prepare $|\phi(x_i)\rangle$
2. Prepare $|\phi(x_j)\rangle$
3. Compute overlap (fidelity)

🔹 Model Training

```python
qsvm = SVC kernel=quantum_kernel.
```

👉 Important:

- Optimization is still **classical**
- Only kernel is quantum

🔹 Result

```python
QSVM Accuracy  0.8859
```

✔ Decent performance
❗ But not necessarily better than classical SVM

---

3. Critical Insight (You need to understand this)

Your assumption might be:

> "Quantum model is better"

❌ Not necessarily

✔ In your setup:

- Very low dimension (2 features)
- Small circuit depth
- No noise simulation

👉 Classical RBF kernel will often perform **equal or better**

---

4. Major Issue in Your Setup

⚠️ Dependency Conflict

[redacted]

Why this matters:

- Can break visualization / backend tools
- Silent failures possible

✔ Fix (recommended)

```bash
numpy==1
```

---

5. Deprecation Warning (Important)

```
[redacted]
```

You are using:

```python
Sampler
```

👉 This will be removed soon

✔️ Better Approach (future-proof)

Use **V2 primitives** (Estimator/Sampler v2 APIs)

---

6. Why Quantum Kernel Works

Classical SVM:

- Uses kernel like:
  - Linear
  - RBF

QSVM:

👉 Uses **Hilbert space mapping**

- Very high-dimensional space
- Hard to simulate classically (in theory)

---

7. When QSVM Actually Helps

QSVM is useful when:

- Data has **complex non-linear structure**
- Feature interactions are important
- Classical kernels fail

---

8. Why Your Accuracy is ~0.88

Let's analyze:

Reasons:

1. Feature reduction (30 → 2) ❗
2. Limited circuit expressibility
3. No hyperparameter tuning
4. No noise model

---

9. Debugging Workflow (important for you)

If accuracy drops:

Check 1:

- Feature scaling correct?

Check 2:

- Feature dimension too small?

👉 Try:

```python
X = X     4
```

Check 3:

- Feature map reps

```python
ZZFeatureMap reps=3
```

Check 4:

- Compare with classical baseline

```python
SVC kernel='rbf'
```

👉 If classical > quantum → no advantage

---

10. Key Concept Summary

Component	Role
Feature Map	Encode data into quantum state
Kernel	Measure similarity
SVM	Find decision boundary

---

11. One-Line Intuition

👉 QSVM =
"Use quantum states to measure similarity instead of classical distance"

---

12. Honest Evaluation (important)

Your implementation is:

✔️ Correct
✔️ Good for demo
❗ Not showing real quantum advantage

---

13. If you want to go deeper (recommended)

I can help you:

1. Build optimized QSVM pipeline

- better dataset
- better encoding

2. Compare:

- QSVM vs RBF vs Polynomial kernel

3. Implement:

- Noise-aware quantum kernel
- Real backend execution

4. Provide:

- Research-paper-level explanation (kernel theory + RKHS)

Before that, tell me:

Do you want:

1. **Better accuracy**
2. **Better understanding**
3. **Research-level implementation**

Pick one direction.